

Big Data Analytics (BDA)

Lab Experiments — Step-by-Step Guide

Complete instructions for absolute beginners

Before You Begin — Initial Setup

This section covers what you need to set up ONE TIME before running any experiments. Do this first.

What is Hadoop?

Hadoop is a software framework that allows processing of large data sets across computers. It uses two things:

1. HDFS (Hadoop Distributed File System) — stores your files
2. MapReduce — processes your files using Mapper and Reducer programs

Step 1 — Open the Terminal

The terminal is a black window where you type commands. Here is how to open it:

1. Press the keyboard shortcut: Ctrl + Alt + T
2. OR: Click the app launcher (bottom-left or top-left icon) → search for 'Terminal' → click it
3. A black/dark window will open. This is where you will type all commands.

 **Note:** Every command in this guide must be typed in the Terminal and then pressed Enter to run.

Step 2 — Check if Hadoop is Running

Type the following command and press Enter:

```
jps
```

You should see something like:

```
12345 NameNode
12346 DataNode
12347 ResourceManager
12348 NodeManager
```

If you see these names, Hadoop is already running. If the terminal just shows a number (like '12345 Jps') and nothing else, start Hadoop with:

```
start-all.sh
```

Wait 10-15 seconds and run jps again to confirm it started.

Step 3 — Create a Working Folder

Create a single folder on your Desktop to keep all your Java files organized:

```
mkdir ~/Desktop/bda_lab
```

This creates a folder called bda_lab on your Desktop. Save ALL your .java files here.

EXPERIMENT 1 — Word Count

Experiment 1: Word Count

Goal: Count how many times each word appears in a text file.

Files needed: WordCountDriver.java, WordCountMapper.java, WordCountReducer.java

How it works (simple explanation)

Mapper: Reads each line of text → splits into individual words → outputs each word with the number 1

Reducer: Collects all the 1s for the same word → adds them up → outputs word with total count

Step 1 — Save the Java Files

1. Open File Manager (the folder icon on your taskbar or desktop)
2. Navigate to Desktop → bda_lab folder
3. Save these 3 files inside bda_lab: WordCountDriver.java, WordCountMapper.java, WordCountReducer.java

Step 2 — Create a Sample Input File

You need a text file to count words from. Create one using the terminal:

```
cd ~/Desktop/bda_lab
echo 'hello world hello hadoop world hello' > input.txt
```

This creates input.txt with the text: hello world hello hadoop world hello

✓ You can also create input.txt using any text editor (like gedit or mousepad). Just type some words and save it as input.txt inside bda_lab folder.

Step 3 — Navigate to Your Folder in Terminal

```
cd ~/Desktop/bda_lab
```

This moves the terminal 'into' your bda_lab folder. All commands from now will run from here.

Step 4 — Compile the Java Files

This converts your .java code into .class files that the computer can run:

```
javac -classpath `hadoop classpath` -d . *.java
```

Explanation of this command:

- javac = Java compiler
- -classpath `hadoop classpath` = tells Java where to find Hadoop libraries
- -d . = put compiled files in current folder
- *.java = compile ALL .java files in this folder

 **Note:** If you see errors like 'cannot find symbol', make sure all 3 java files are in the same folder.

Step 5 — Create a JAR File

A JAR file bundles all your compiled .class files into one package:

```
jar -cvf wordcount.jar *.class
```

This creates a file called wordcount.jar in your bda_lab folder.

Step 6 — Upload Input File to HDFS

Hadoop reads files from its own file system (HDFS). You need to copy your input.txt there:

```
hdfs dfs -mkdir /wc_input  
hdfs dfs -put input.txt /wc_input
```

- mkdir /wc_input = creates a folder called wc_input inside HDFS
- -put input.txt /wc_input = copies input.txt from your computer into HDFS

Step 7 — Run the Word Count Program

```
hadoop jar wordcount.jar WordCountDriver /wc_input /wc_output
```

Explanation:

- hadoop jar = run a hadoop job using a jar file
- wordcount.jar = the jar file to run
- WordCountDriver = the main class (entry point) of the program
- /wc_input = input folder in HDFS (where your input.txt is)
- /wc_output = output folder in HDFS (Hadoop will create this automatically)

 **Note:** The output folder /wc_output must NOT already exist. If it does, delete it: `hdfs dfs -rm -r /wc_output`

Step 8 — View the Output

```
hdfs dfs -cat /wc_output/part-r-00000
```

Expected output (based on our sample input.txt):

```
hadoop      1  
hello 3  
world 2
```

The number after the word is how many times it appeared in the input file.

✅ Congratulations! You ran your first Hadoop MapReduce program.

EXPERIMENT 2 — Find Maximum Number

Experiment 2: Find Maximum Number

Goal: Find the largest number from a file containing numbers (one number per line).

Files needed: MaxDriver.java, MaxMapper.java, MaxReducer.java

How it works (simple explanation)

Mapper: Reads each number → outputs it with the key 'max'

Reducer: Receives all numbers under key 'max' → keeps track of the largest one → outputs it

Step 1 — Save the Java Files

1. Save MaxDriver.java, MaxMapper.java, MaxReducer.java inside ~/Desktop/bda_lab

 **Note:** *MaxReducer.java has a capital J in the extension — this is fine, save it exactly as given.*

Step 2 — Create a Sample Input File

Create a file with numbers, one per line:

```
cd ~/Desktop/bda_lab
printf '45\n23\n78\n12\n99\n56\n' > numbers.txt
```

This creates numbers.txt with the numbers: 45, 23, 78, 12, 99, 56 — each on a separate line.

✓ You can also open any text editor, type numbers (one per line), and save as numbers.txt in bda_lab.

Step 3 — Compile the Java Files

In terminal, make sure you are in bda_lab folder first:

```
cd ~/Desktop/bda_lab
javac -classpath `hadoop classpath` -d . *.java
```

Step 4 — Create the JAR File

```
jar -cvf maxnum.jar *.class
```

Step 5 — Upload Input File to HDFS

```
hdfs dfs -mkdir /max_input  
hdfs dfs -put numbers.txt /max_input
```

Step 6 — Run the Max Number Program

```
hadoop jar maxnum.jar MaxDriver /max_input /max_output
```

 **Note:** If `/max_output` already exists from a previous run, delete it first: `hdfs dfs -rm -r /max_output`

Step 7 — View the Output

```
hdfs dfs -cat /max_output/part-r-00000
```

Expected output:

```
max    99
```

This means the maximum number found in your file is 99.

EXPERIMENT 3 — Calculate Average

Experiment 3: Calculate Average

Goal: Calculate the average (mean) of all numbers in a file.

Files needed: AvgMapper.Java, AvgReducer.java

 **Note:** This experiment has NO Driver file in the given code. You need to create one yourself OR reuse the MaxDriver pattern. Instructions below cover both options.

How it works (simple explanation)

Mapper: Reads each number → outputs it with the key 'avg'

Reducer: Collects all numbers → adds them all up → divides by count → outputs the average

Option A — Create an AvgDriver.java file

Since no AvgDriver.java was provided, you need to create one. Open a text editor (gedit/mousepad) and create a new file called AvgDriver.java with this content:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AvgDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Average Number");
        job.setJarByClass(AvgDriver.class);
        job.setMapperClass(AvgMapper.class);
        job.setReducerClass(AvgReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Save this file as AvgDriver.java inside ~/Desktop/bda_lab

Step 1 — Prepare Input File

You can reuse the same numbers.txt from Experiment 2, or create a new one:

```
cd ~/Desktop/bda_lab
printf '10\n20\n30\n40\n50\n' > avgnumbers.txt
```

Step 2 — Compile

```
cd ~/Desktop/bda_lab
javac -classpath `hadoop classpath` -d . *.java
```

Step 3 — Create the JAR File

```
jar -cvf average.jar *.class
```

Step 4 — Upload to HDFS

```
hdfs dfs -mkdir /avg_input
hdfs dfs -put avgnumbers.txt /avg_input
```

Step 5 — Run the Average Program

```
hadoop jar average.jar AvgDriver /avg_input /avg_output
```

Step 6 — View the Output

```
hdfs dfs -cat /avg_output/part-r-00000
```

Expected output (for 10, 20, 30, 40, 50):

```
avg    30.0
```

The average of $10+20+30+40+50 = 150 \div 5 = 30.0$

EXPERIMENT 4 — Filter Data (HR Department)

Experiment 4: Filter Data

Goal: Filter rows from a CSV file and keep only rows where the department is 'HR'.

Files needed: FilterDriver.java, FilterMapper.java

How it works (simple explanation)

Mapper: Reads each line → splits by comma → checks if 3rd column (index 2) is 'HR' → if yes, outputs the full line

Reducer: None (setNumReduceTasks(0) means no reducer is used — data goes directly to output)

Understanding the Input Format

The input CSV file must have at least 3 columns separated by commas. The 3rd column (index 2) is the department. Example:

```
101,Alice,HR
102,Bob,IT
103,Charlie,HR
104,David,Finance
105,Eve,HR
```

The program will keep only the rows where column 3 = HR (Alice, Charlie, Eve).

Step 1 — Create the Input CSV File

Open a text editor and create a file called employees.csv with the content above. Save it in ~/Desktop/bda_lab

OR use the terminal:

```
cd ~/Desktop/bda_lab
printf
'101,Alice,HR\n102,Bob,IT\n103,Charlie,HR\n104,David,Finance\n105,Eve,HR\n'
> employees.csv
```

Step 2 — Compile

```
cd ~/Desktop/bda_lab
```

```
javac -classpath `hadoop classpath` -d . FilterDriver.java
FilterMapper.java
```

 **Note:** Here we compile only 2 specific files instead of *.java. This avoids errors if you have other experiments' files in the same folder with incomplete driver files.

Step 3 — Create the JAR File

```
jar -cvf filter.jar FilterDriver.class FilterMapper.class
```

Step 4 — Upload to HDFS

```
hdfs dfs -mkdir /filter_input
hdfs dfs -put employees.csv /filter_input
```

Step 5 — Run the Filter Program

```
hadoop jar filter.jar FilterDriver /filter_input /filter_output
```

Step 6 — View the Output

```
hdfs dfs -cat /filter_output/part-m-00000
```

 **Note:** Notice it is part-m-00000 (m for mapper) not part-r-00000 (r for reducer) because this job has no reducer.

Expected output:

```
101,Alice,HR
103,Charlie,HR
105,Eve,HR
```

Only the HR employees are shown. The IT and Finance rows are filtered out.

QUICK REFERENCE — Commands Cheat Sheet

Quick Reference — All Commands at a Glance

Hadoop Management

```
jps                # Check if Hadoop is running
start-all.sh      # Start Hadoop
stop-all.sh       # Stop Hadoop
```

HDFS Commands

```
hdfs dfs -ls /      # List files in HDFS root
hdfs dfs -mkdir /folder # Create folder in HDFS
hdfs dfs -put file /folder # Upload file to HDFS
hdfs dfs -cat /path/file # View file content
hdfs dfs -rm -r /folder # Delete folder from HDFS
```

Compile and Run Pattern (for every experiment)

```
# Step 1: Go to your folder
cd ~/Desktop/bda_lab

# Step 2: Compile all java files
javac -classpath `hadoop classpath` -d . *.java

# Step 3: Create jar
jar -cvf myprogram.jar *.class

# Step 4: Upload input to HDFS
hdfs dfs -mkdir /input_folder
hdfs dfs -put inputfile.txt /input_folder

# Step 5: Run
hadoop jar myprogram.jar DriverClassName /input_folder /output_folder

# Step 6: View output
hdfs dfs -cat /output_folder/part-r-00000
```

Common Errors and Fixes

Error	Fix
Output directory already exists	Run: <code>hdfs dfs -rm -r /output_folder_name</code>
cannot find symbol	All .java files must be in the same folder
Connection refused	Hadoop is not running. Run: <code>start-all.sh</code>
No such file or directory	Check you are in <code>~/Desktop/bda_lab</code> with: <code>pwd</code>

Good luck with your lab exam tomorrow! 🍀